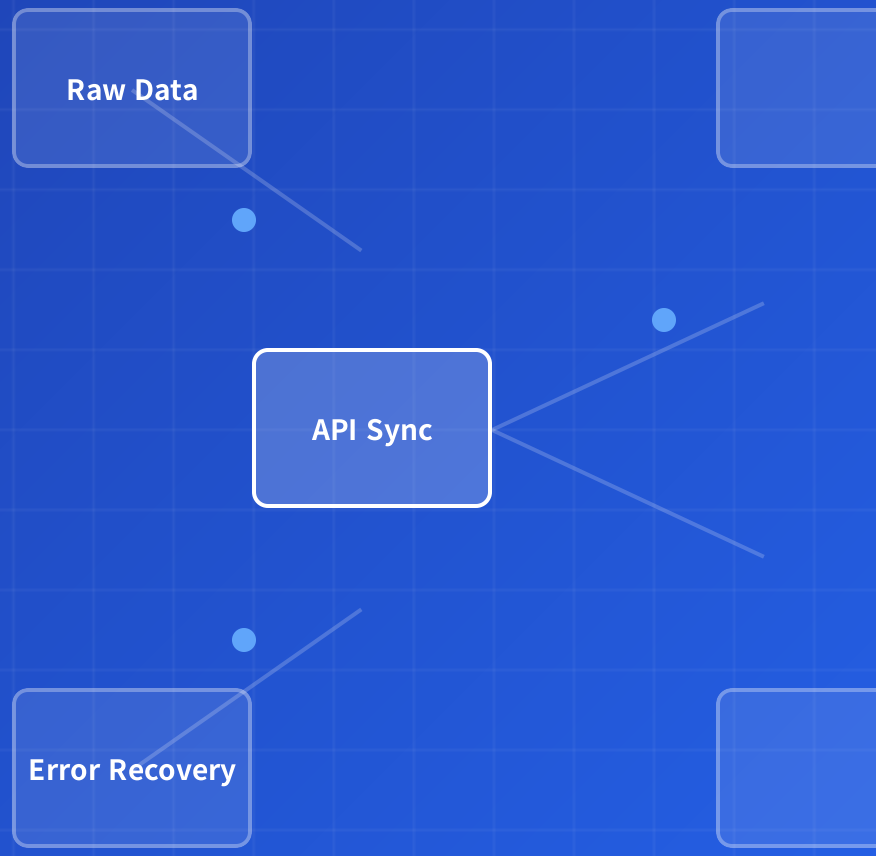
한국 주소 데이터 전처리 자동화 파이프라인

부산대학교병원 코로나19 환자 데이터 80만 건 전처리 프로젝트

API 자동화 파이프라인 및 2단계 복구 시스템 구축으로
일일 데이터 처리량 **3,200% 향상**, 80만 건 감염자 주소 데이터 **100% 표준화** 달성

Python | Juso API | Batch Processing | Error Recovery | Data Pipeline

Data Engineering Portfolio | 2024



PROJECT BACKGROUND

80만 건 처리를 가로막는 두 가지 장벽

프로젝트 배경 및 요구사항

2024년 부산대학교병원에서 의뢰한 코로나19 감염 환자 데이터 약 **80만 건**의 전처리 과정에서 시작된 프로젝트입니다. 최종 수요자인 연구자 및 정책가의 요구사항은 도로명 주소에서 **행정구역(시/도, 시/군/구, 읍/면/동) 정보를 100% 정확도로 추출**하는 것이었습니다.

■ 기술적 장벽

도로명 주소와 지번 주소의 **체계 차이**로 인해 단순 정규식만으로는 행정구역 변환이 불가능했습니다. 비정형 주소 데이터를 표준 체계로 변환하는 별도의 자동화 수단이 필요했습니다.

■ 시간적 장벽

수동 웹검색 방식으로는 일일 최대 처리량이 **5,000건**에 불과하여, 전체 80만 건을 처리하는 데 **약 160일**이 소요될 것으로 예상되었습니다. 이는 마감 기한 내 완수가 물리적으로 불가능한 상황이었습니다.

결론: 대규모 데이터를 안정적으로 처리할 수 있는 자동화 파이프라인 구축이 필수적

RESEARCH & DISCOVERY

해결책 발견: Juso API 분석으로 자동화 가능성 확인

조사 대상: 행정안전부 주소정보누리집 Juso API 및 도로명 주소 체계 표준 가이드

핵심 발견 1: API 활용 가능성

Juso API는 대한민국 도로명 주소 ↔ 지번 주소 매칭 데이터베이스를 API 형식으로 제공합니다. 비정형 도로명 주소를 표준 행정구역 데이터로 **자동 변환하는 핵심 수단**으로 활용 가능함을 확인했습니다.

핵심 발견 2: 잔여 오류 패턴 파악

상세주소(층·호수), 통/반 정보 등 비표준 형식이 포함될 경우 API 요청이 실패함을 확인했습니다. 정규식으로 사전 정제해도 다양한 오류가 잔존하여 **사람이 직접 개입하는 수동 정제 단계**가 필요했습니다.

유형	원본 주소 예시 (오류 원인)	처리 방법
상세주소 포함	부산광역시 동구 홍곡남로 6 **동신탕 3층**	건물명·층수 제거
통/반 정보 포함	부산광역시 사상구 백양대로433번길 9 **7통 4반**	통/반 정보 제거
정규식 처리 오류	부산광역시 해운대구 센텀3로 26 **3009호**	호수 정보 제거
폐지된 주소	도로명 변경·폐지된 주소 (API 기본 검색 실패)	"폐지된 주소 포함" 옵션으로 재검색

TECH STACK

기술 스택: 데이터 안정성을 보장하는 핵심 설계

Python 3.x
핵심 언어

Pandas
데이터 처리

Requests
REST API 통신

Logging & tqdm
진행 모니터링

Core Architecture

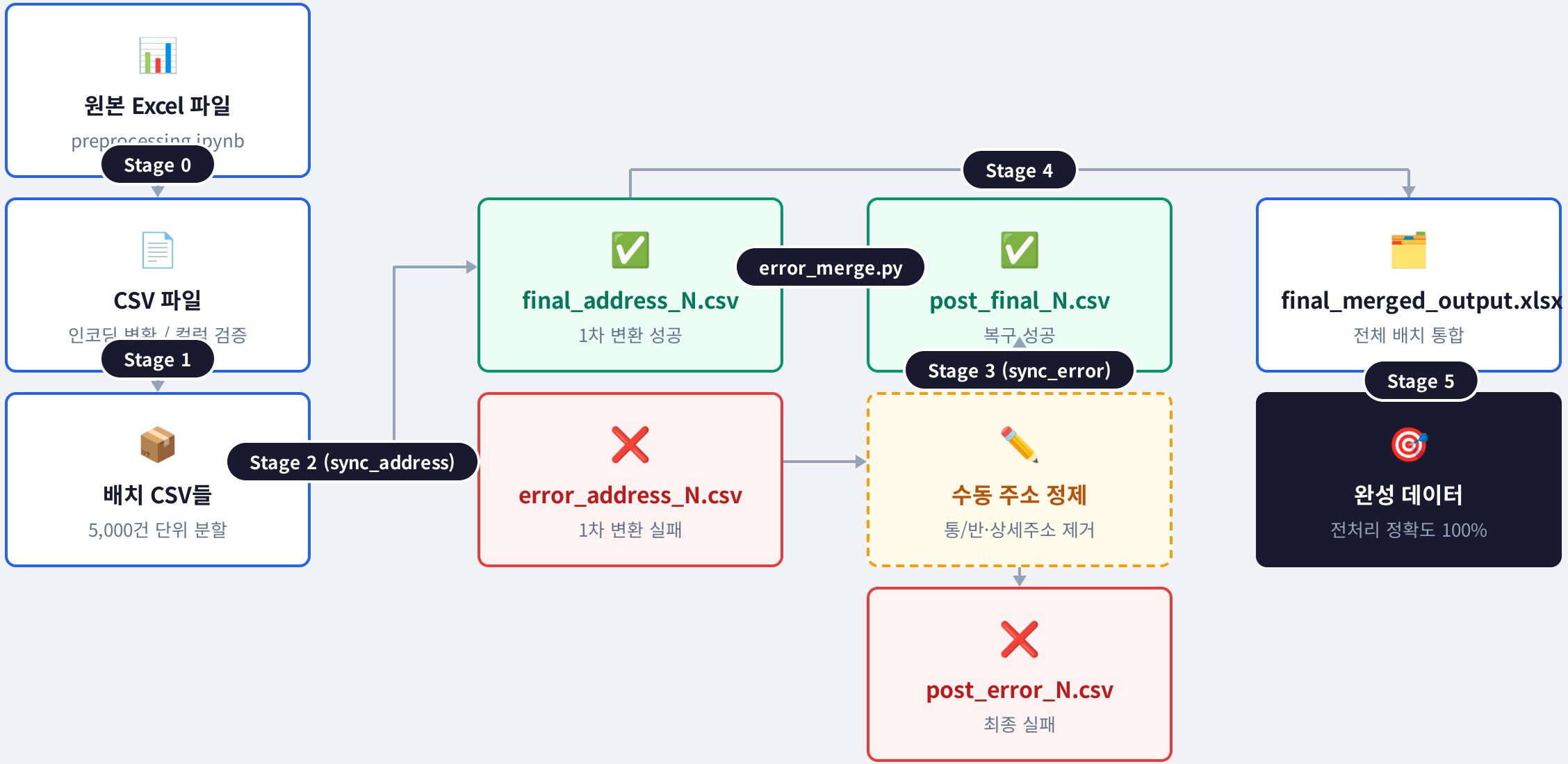
- 대용량 데이터의 안정적인 처리를 위한 **배치 분할 시스템**
- 네트워크 및 API 서버 부하를 고려한 **지능형 통신 설계**
- 실시간 진행 상황 모니터링 및 **로그 기록 체계** 구축

Key Capabilities

- **3단계 에러 분류** 및 자동 재시도 메커니즘
- 원본 인덱스 보존을 통한 **무결성 보장 데이터 병합**
- **2단계 에러 복구** (Stage 2-3) 아키텍처 핵심 기능 구현

ARCHITECTURE 1

6단계 파이프라인 전체 구조



TECHNICAL DECISIONS

지능형 API 재시도 로직: 3단계 에러 분류로 안정성 확보

Technical Decisions

설정 항목	선택 근거 및 전략
Retries (3회)	일시적 네트워크 오류 대응 및 영구적 실패 판단 기준
Delay (3초)	API 서버 부하 방지 및 Rate Limit 회피를 위한 대기
Timeout (50초)	대용량 요청 시 발생할 수 있는 응답 지연 고려
hstryYn ("Y")	폐지된 주소까지 검색하여 데이터 변환 성공률 극대화

Error Handling Strategy

01. HTTP 에러

서버 연결 실패 또는 500번대 에러 발생 시 **자동 재시도**를 수행하여 일시적 장애를 극복합니다.

02. API 에러

유효하지 않은 주소나 결과 없음 응답 시 에러 로깅 후 재시도하여 데이터 정합성을 확인합니다.

03. 네트워크 에러

RequestException 등 통신 예외 발생 시 타임아웃 관리 및 재시도 로직을 가동합니다.

BATCH STRATEGY

5,000건 배치 전략: 처리 효율과 장애 복구의 최적 균형점

대용량 데이터 처리 시 발생할 수 있는 메모리 부하와 네트워크 타임아웃을 방지하기 위해 데이터를 **5,000건 단위**로 분할하는 배치 전략을 채택했습니다.

Optimization Benefits

- I/O 오버헤드와 실패 시 재처리 비용의 최적 균형점
- 각 배치별 중간 결과물 저장으로 진행 상황 실시간 모니터링
- 네트워크 불안정 시 최대 손실 범위를 관리 가능한 수준으로 제한

결정 근거: API 호출 1건당 약 0.3~0.5초 소요를 고려할 때, 5,000건은 작업 연속성과 시스템 안정성을 동시에 확보할 수 있는 최적의 단위입니다.

배치 크기	처리 시간	장애 시 최대 손실 및 리스크	채택 여부
1,000건	~8분	8분 / 파일 수 과다로 인한 I/O 오버헤드	—
5,000건	**~33분**	**33분 (최적의 관리 범위)**	**✓ 채택**
10,000건	~67분	67분 / 재처리 비용 및 리스크 증가	—
50,000건+	~5.5시간	네트워크 타임아웃 위험 극대화	—

ERROR RECOVERY PROCESS

에러 복구 프로세스 상세: 자동+수동 조합으로 100% 달성

Stage 01

1차 자동 API 처리

(sync_address.py)

전체 80만 건 데이터를 대상으로 표준 API 호출을 수행합니다. 성공 데이터는 즉시 저장하고, 실패 데이터는 **원본 인덱스를 유지**한 채 별도로 분류합니다.

Stage 02

실패 주소 수동 정제

(Data Cleaning)

API 검색 실패의 주요 원인인 **'통/반 정보'** 및 **'상세주소'** 등 불필요한 정보를 제거합니다. 비정형 텍스트를 API가 인식 가능한 **최소 단위의 도로명 주소**로 정제합니다.

Stage 03

2차 복구 API 처리

(sync_error.py)

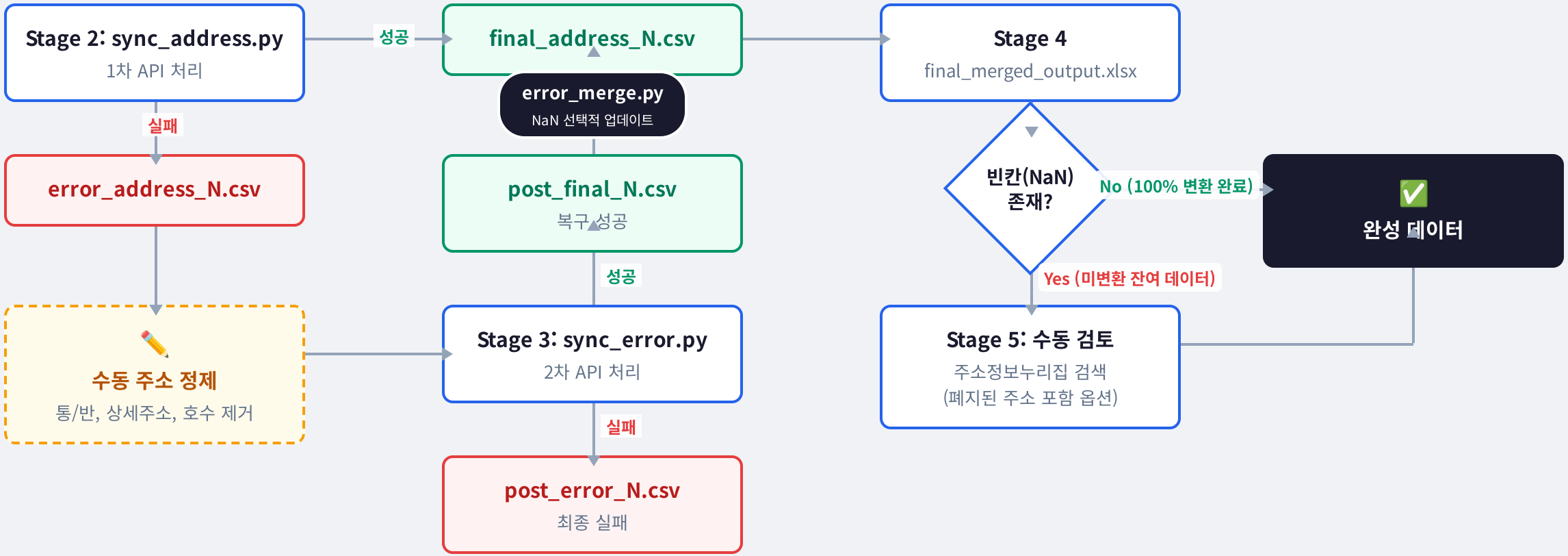
정제된 주소를 바탕으로 재호출을 수행합니다. **error_merge.py**를 통해 기존 데이터의 빈칸(NaN)만 선택적으로 업데이트하여 무결성을 유지합니다.

Selective Update Logic

pd.isna() 체크를 통해 이미 성공한 데이터를 보호하며, 원본 인덱스 기반의 정밀 매칭으로 최종 데이터 완성도를 **99% 이상**으로 극대화합니다. 잔여 NaN 값은 Stage 5 수동 검토를 통해 **100% 완성**합니다.

ARCHITECTURE 2

아키텍처 — 2단계 에러 복구 시스템



설계 핵심 원칙

1. pd.isna() 체크 → 이미 성공한 데이터 덮어쓰기 방지 **2. 원본 인덱스 보존** → 배치 분할 이후에도 정확한 위치에 결과 병합 **3. 자동+수동 조합** → 100% 정확도 달성

ACTIONS TAKEN

실행한 액션: 파이프라인 구축부터 팀 표준화까지

Action 1
자동 변환 파이프라인 설계

- 수작업 한계 극복을 위해 Juso API를 활용한 **6단계 배치 처리 파이프라인** 구축
- **5,000건 단위 배치 분할**로 네트워크 장애 시 최대 손실 범위를 33분으로 제한
- **API 재시도 로직**(3회, 3초 간격)으로 일시적 네트워크 오류 자동 복구

Action 2
2단계 에러 복구 시스템 구축

- **1차 복구:** API 실패 주소를 도로명 주소 표준 가이드에 맞게 수동 정제 후 재요청
- **2차 복구:** 최종 병합 후 잔여 NaN 값을 주소정보누리집에서 검색하여 수동 입력
- 결과: 전처리 정확도 **100% 달성**

Action 3
팀 협업 표준화 (가이드라인)

- 노션(Notion)에 주소 정제 규칙, API 에러 대응 방법, 파이프라인 사용법 **상세 문서화**
- 동영상 가이드 촬영 및 배포하여 팀원 **5명**이 동일한 표준 프로세스로 병렬 작업 수행

RESULTS

성과: 일일 처리량 3,200% 향상, 80만 건 100% 표준화

+3,200%	약 32배	100%	80만 건
일일 처리량 향상률	공정 시간 단축	전처리 정확도	처리 완료 데이터

지표	기존 (수동 방식)	개선 후 (자동화 파이프라인)
일일 처리량	5,000건 (수동 웹검색)	**16만 건**
전체 처리 예상 기간	약 160일	**약 5일**
전처리 정확도	미측정	**100%**

핵심 기술 포인트 5가지

1. **배치 분할 처리** — 5,000건 단위 분할로 장애 시 손실 최소화 (최대 33분)
2. **API 재시도 로직** — 3회 재시도 + 3초 대기로 일시적 네트워크 오류 자동 복구
3. **2단계 에러 복구** — 자동(정규식 정제) + 수동(사람 개입) 조합으로 100% 정확도 달성
4. **인덱스 기반 병합** — 배치 분할 이후에도 원본 인덱스 보존으로 데이터 정합성 유지
5. **팀 협업 표준화** — 노션 가이드라인 + 동영상 매뉴얼로 5인 팀 생산성 극대화